REPORT

Degree : B.Tech

Branch : CSE

Section : C

Date Of Submission : 30th April, 2026

BUS ROUTE OPTIMIZER

A Geospatial Web Mapping Tool for Route Optimization

GROUP DETAILS		
S.NO	**REGISTRATION NUMBER**	**NAME OF THE STUDENT**
1.	AP24110011822	Devakeenandan V
2.	AP24110011876	Vijay Hemanth A
3.	AP24110011883	Bhargav Sriram A
4.	AP24110011873	Gowtham Biyani
5.	AP24110011875	Rahaman Shareef

ACKNOWLEDGEMENT

We would like to express our deepest and most heartfelt gratitude to our beloved Coding Skills faculty, **Mr. Prakash Sir**, whose extraordinary dedication, patience, and passion for teaching have been the true backbone of this entire project. His classes were never just lectures — they were life-changing sessions that transformed the way we think, code, and solve problems.

Every algorithm we implemented, every line of code we wrote, and every challenge we overcame during the development of this Bus Route Optimizer traces back to the strong programming foundation that Prakash Sir so carefully and brilliantly built in us. His unique ability to explain even the most complex concepts in the simplest possible way is a gift that very few teachers possess, and we feel incredibly lucky to have been his students.

Sir, your encouragement never let us give up. Whenever we were stuck, your words pushed us forward. Whenever we doubted ourselves, your belief in us gave us the confidence to keep going. This project is as much yours as it is ours.

We also sincerely thank our college management, our Head of Department, and all our faculty members for their constant support and for providing us with an environment that encourages creativity and growth. We are grateful to our parents and families for standing by us through every late night and every challenging moment.

Finally, we thank our teammates, our peers, and the open-source community — the developers of Python, Flask, and Leaflet.js — whose tools made this project technically possible.

"A great teacher does not just teach you what to think — he teaches you how to think. Thank you, Prakash Sir, for being that teacher for us."

ABSTRACT

The **"Bus Route Optimizer"** is a modern, interactive web application designed to solve the classic Traveling Salesperson Problem (TSP) in the context of urban bus routing. In today's rapidly growing cities, efficient public transportation is no longer a luxury — it is an absolute necessity. This project directly addresses that need by providing a smart, visual, and computational platform that calculates the shortest possible route between multiple geographic bus stops.

The application leverages real-world geospatial data using the **Haversine formula**, which accurately computes geodesic distances between coordinates by accounting for the curvature of the Earth — going far beyond simple straight-line estimations that traditional planning tools rely on.

At its core, the system implements and compares two powerful algorithms. The first is the **Greedy Nearest Neighbour heuristic**, which delivers rapid, approximate routing solutions and scales effortlessly to handle hundreds of stops in milliseconds. The second is the **HeldKarp Dynamic Programming algorithm**, a mathematically exact method that guarantees the absolute shortest route, though it is practically suited for networks of up to 20 stops due to its exponential computational complexity.

Built with a **Python and Flask** backend serving as the API engine, and a **Leaflet.js** powered frontend for rich, interactive map visualization, the system allows users to drop bus stop markers on a live map, search for real-world addresses via a geocoder, designate a depot, and instantly visualize the computed optimal route as a polyline drawn across the map.

The project successfully demonstrates the real-world trade-off between **speed and optimality** — giving transit planners and developers a powerful tool to understand, visualize, and minimize travel distances in urban transit networks.

.

TABLE OF CONTENTS

1. Introduction 4
2. Problem Statement 4
3. Objective 5
4. System Architecture & Technologies 5
5. Algorithms Used 6
6. Implementation Details 6
7. Conclusion 7
8. Future Scope 7

1. INTRODUCTION

In an increasingly urbanized world, efficient public transportation is critical. A major challenge in transit management is minimizing the travel distance and time for vehicles making multiple stops. The Bus Route Optimizer provides a visual and computational platform to address this issue. It allows users to plot locations on an interactive map, search for real-world addresses, and instantly compute the shortest possible path connecting all points, ultimately returning to a designated depot.

2. PROBLEM STATEMENT

Traditional route planning often relies on estimations or Euclidean (straight-line) distances on a flat plane, which fails to account for the Earth's curvature. Furthermore, determining the absolute shortest path for a given set of stops (the Traveling Salesperson Problem) is an NPhard computational challenge. Transit operators need a tool that not only calculates accurate real-world distances but also provides a balance between rapid route estimation for

large numbers of stops and finding the mathematically optimal route for smaller, intricate networks.

3. OBJECTIVE

The primary objectives of this project are to:

- Develop an interactive web-based map interface for easy visualization and placement of bus stops.
- Implement the Haversine formula to compute accurate geodesic distances between real-world coordinates.
- Provide a rapid routing solution using the Greedy algorithm.
- Provide an optimal routing solution using the Held-Karp (Dynamic Programming) algorithm.
- Allow users to compare the performance, path length, and execution time of both approaches.

4. SYSTEM ARCHITECTURE & TECHNOLOGIES

The project follows a client-server architecture. The major components are described below:

Component	Details
Frontend	HTML, Vanilla CSS, and JavaScript. Utilizes Leaflet.js for rendering the interactive map and a Geocoder plugin for address lookup.
Backend	Python and Flask, serving as the API and web server.
Distance Calculation	The Haversine formula is implemented in Python to handle geospatial data (latitude and longitude) and calculate accurate distances in kilometers.

5. ALGORITHMS USED

5.1 Greedy Algorithm (Nearest Neighbour)

Starting from the depot, the algorithm continuously selects the nearest unvisited stop until all stops are visited, then returns to the depot.

Property	**Greedy (Nearest Neighbour)**	**Dynamic Programming (Held-Karp)**
Concept	Selects nearest unvisited stop at each step	Solves TSP using bitmasking & memoization
Time Complexity	$O(n^2)$	$O(n^2 \cdot 2^n)$
Space Complexity	$O(n)$	$O(n \cdot 2^n)$
Optimality	Heuristic (not guaranteed)	Mathematically optimal
Scalability	Handles hundreds of nodes easily	Practical limit ~20 nodes

5.2 Dynamic Programming (Held-Karp)

An exact algorithm that solves the Traveling Salesperson Problem by breaking it down into overlapping subproblems. It uses bitmasking to keep track of visited nodes and memorizes the shortest paths to subsets of nodes. It is mathematically guaranteed to find the absolute shortest route. Due to its exponential complexity, its use is practically limited to networks of about 20 stops or fewer before computational time and memory usage become prohibitive.

6. IMPLEMENTATION DETAILS

- Interactive Map: Users can click on the map to drop markers representing bus stops, or use the integrated search bar to find specific locations geographically.

- Depot Selection: Any node added to the map can be explicitly set as the starting bus depot.
- Route Visualization: Once calculated, the route is drawn directly onto the Leaflet map as a polyline, visually connecting the stops in sequence.
- Safety Constraints: To prevent server crashes or timeouts, the backend actively restricts the Held-Karp algorithm from running on inputs exceeding 20 nodes.

7. CONCLUSION

The Bus Route Optimizer successfully bridges the gap between theoretical computer science algorithms and practical geographic application. By implementing the Haversine formula alongside both heuristic and optimal TSP algorithms, the application effectively demonstrates the trade-offs between computational speed and route optimality. The intuitive Leaflet.js interface makes complex mathematical routing accessible to end-users, proving the system's viability as a foundation for real-world transit planning.

8. FUTURE SCOPE

Future enhancements could include:

- Integration with live traffic data APIs (e.g., Google Maps Directions API or OpenStreetMap routing engines) to calculate routes based on real-time drive time rather than just geographic distance.
- Implementation of more advanced heuristic algorithms, such as Simulated Annealing or Genetic Algorithms, to find near-optimal solutions for large datasets without the exponential time cost of Held-Karp.
- Multi-vehicle routing to solve the broader Vehicle Routing Problem (VRP).